Harmony ONE

Smart contract audit / Ethereum ERC-20

08 September 2026

REVIEWED SOURCE	HarmonyONEToken.sol / 71 lines / 2,970 bytes
SHA-256	`9ad09df755ba694e3f7016cdeeafeb9456cce11990f9b0f74cf78e406d95a4ea`

No security defects identified.

Fixed supply of 26,271,000,000 ONE (18 decimals). No privileged role, upgrade mechanism, or post-deployment mint/burn path. No source change recommended within the reviewed scope.

0 CRITICAL	0 HIGH	0 MEDIUM	0 LOW	2 INFO

INFORMATIONAL OBSERVATIONS

01 **ETH can be forced into the contract**

Ordinary ETH transfers revert, but **SELFDESTRUCT** can credit ETH without executing this contract. Forced ETH cannot be recovered; token accounting is unaffected.

02 **Zero-address errors depend on check order**

transferFrom(address(0), to, 0) reverts with **ERC20InvalidApprover(0)**; a positive amount fails allowance first. Zero-address checks therefore do not always determine the first error. This matches OpenZeppelin v5.6.1.

INDEPENDENT VALIDATION

Byte-identical ERC-20 behavior validated against OpenZeppelin v5.6.1 across all 480,000 tested operations. Return/revert bytes and event topics/data matched exactly, as did resulting balances and allowances. **20,000 sequences; 33 tests passed.**

Verified: fixed supply, balance conservation, ERC-20 interface and custom errors, allowance behavior, arithmetic safety and two-mapping storage. Functional OpenZeppelin equivalence assumes sufficient gas.

BUILD & DEPLOYMENT

solc 0.8.36 / Prague / optimizer 200 / viaIR off. No applicable listed compiler bug; emitted opcodes are mainnet-supported. Runtime **1,465 B**; init code **1,702 B**. Executable runtime matched the project's expected hash; full bytecode hashes differed. Resolve build reproducibility and verify the exact bytecode before deployment.

Scope: identified source only. Supply policy, treasury Safe configuration, distribution and deployed bytecode are excluded. Testing is not a formal proof or a guarantee of safety.

Fable 5.1 Max, GPT-6 Astra Ultra, and Daybreak Blue were used for preparing and conducting this audit.

hiddenstate.xyz participates in OpenAI's Trusted Access for Cyber (TAC) and Anthropic's Cyber Verification Program.

Harmony ONE - smart contract audit

Auditor: hiddenstate.xyz
Date: 8 September 2026
Method: Source review, independent EVM tests and differential fuzzing

Fable 5.1 Max, GPT-6 Astra Ultra, and Daybreak Blue were used for preparing and conducting this audit.

hiddenstate.xyz participates in OpenAI's Trusted Access for Cyber (TAC) and Anthropic's Cyber Verification Program.

Conclusion

No critical, high, medium or low security defects identified. Two informational observations.

The reviewed contract implements a fixed supply of **26,271,000,000 ONE with 18 decimals**, assigned once to the constructor recipient. Its runtime contains no privileged role, upgrade mechanism, mint or burn path. Transfers conserve supply, finite allowances are consumed correctly, and unlimited allowances remain unchanged. No source change is recommended within the reviewed scope.

Byte-identical ERC-20 behavior was validated against OpenZeppelin Contracts v5.6.1 across all 480,000 tested operations. Success/failure outcomes, return/revert bytes and event topics/data matched exactly, as did resulting balances and allowances. This functional comparison assumes sufficient gas and compares each implementation's events at its own contract address. It does not claim identical bytecode or gas consumption, and testing is not a formal proof of every possible execution.

Scope and source identity

Item	Reviewed value
Source	`HarmonyONEToken.sol`, 71 lines, 2,970 bytes, no imports
SHA-256	`9ad09df755ba694e3f7016cdeeafeb9456cce11990f9b0f74cf78e406d95a4ea`
Source keccak256	`0x2d9aeb4ff4e3fa3188cfcc98add4de73e1fce24228f25cc81f26d021e6fb84f9`
Compiler	`0.8.36+commit.8a079791`
Settings	`evmVersion=prague`, optimizer enabled, 200 runs, `viaIR=false`, IPFS/CBOR metadata
Reference	OpenZeppelin Contracts v5.6.1 ERC20, constructor mint of the same supply to the same recipient
Intended deployment	Ethereum mainnet, chain ID 1; non-zero treasury Safe

The source identity above was independently verified. Supply policy, treasury Safe configuration, distribution, deployment tooling and deployed bytecode are outside this review. No mainnet deployment or treasury address was verified. The findings concern the identified source and independently compiled test artifacts.

Informational observations

I-01 - ETH can be forced into the contract

Severity: Informational. **Location:** Contract-wide.

Ordinary calls transferring ETH revert because no entry point is payable and there is no `receive` or `fallback`. This does not imply that the account's ETH balance must remain zero. Another contract can execute `SELFDESTRUCT` with this token as beneficiary without calling the token. A deterministic deployment address can also be funded before creation. The deployed token can therefore hold ETH, and has no withdrawal path. This does not affect ERC-20 balances or supply. EIP-6780, Solidity security considerations.

Reproduction: Deploy the token; create a helper with 1 ETH whose payable constructor executes `selfdestruct(payable(address(token)))`; observe `address(token).balance == 1 ether` while token accounting remains unchanged. A separate test predicts the next `CREATE` address from the deployer nonce, initializes its balance to 2 ETH with `vm.deal`, then verifies that construction retains this pre-existing ETH.

Recommendation: Describe the behavior as: "No payable entry point; ordinary ETH transfers revert. Forced or pre-funded ETH may remain irrecoverable." No source change is required for the token design.

I-02 - Zero-address errors depend on check order

Severity: Informational. **Location:** Source lines 40-48, 52-57 and 65-67.

Zero-address restrictions do not guarantee that a particular zero-address error is always returned: an earlier check can fail first. `transferFrom` checks and consumes allowance before `_transfer`; finite allowance consumption calls `_approve`, which checks owner before spender. This intentionally matches OpenZeppelin v5.6.1. Reference implementation.

Call / precondition	Actual first error
`transferFrom(address(0), to, 0)` with untouched zero-owner allowance	`ERC20InvalidApprover(address(0))`
`transferFrom(address(0), to, 1)` with zero allowance	`ERC20InsufficientAllowance(caller, 0, 1)`
`transferFrom(holder, address(0), 1)` with zero allowance	`ERC20InsufficientAllowance(caller, 0, 1)`
Same call with sufficient allowance	`ERC20InvalidReceiver(address(0))`; allowance write rolls back
`approve(address(0), value)` under a zero-caller simulation	`ERC20InvalidApprover(address(0))`

Reproduction: The independent boundary suite asserts each case and exact revert data against the reference. Zero-caller cases use EVM test impersonation; ordinary signed transactions do not originate from the zero address.

Recommendation: Document error precedence when describing zero-address restrictions. Preserve the existing check order to maintain OpenZeppelin behavior.

Verified contract properties

Property	Verdict	Evidence and qualification
Fixed supply and balance conservation	Holds	The 29-digit constant is `26_271_000_000_000_000_000_000_000_000`, equal to `26_271_000_000 * 10**18`. Constructor lines 25-27 reject zero and allocate exactly this amount. Source reasoning and boundary/differential tests establish conservation.
No issuance or destruction after deployment	Holds	Only `_transfer` writes balances at runtime. No owner, role, external call, delegatecall, selfdestruct, assembly, fallback, receive or upgrade entry point exists. Construction is the sole issuance event.
Standard ERC-20 interface	Holds	`name`, `symbol`, `decimals`, `totalSupply`, `balanceOf`, `allowance`, `transfer`, `approve` and `transferFrom` expose standard selectors and encodings. Transfer/Approval topics and indexed fields match. All three state-changing functions return `true` on success and revert on failure.
Standard ERC-6093 errors	Holds	All six error signatures/selectors match IERC20Errors. Branch ordering and boundary tests match the reference's exact error arguments.
Zero-address restrictions	Holds with caveat	Transfers reject zero senders/recipients; approvals reject zero owners/spenders; construction rejects a zero recipient. The first error depends on allowance and owner/spender check precedence, as detailed in I-02.
OpenZeppelin allowance semantics	Holds	Allowance precedes balance; finite subtraction is guarded, maximum allowance is untouched, spending emits only Transfer, and approve overwrites and emits Approval. Failed transfers roll allowance changes back.
Arithmetic safety	Holds	Both unchecked subtractions follow lower-bound checks. Checked recipient addition cannot overflow in reachable state, including self-transfers.
Functional equivalence to OpenZeppelin v5.6.1	Holds with caveat	Source paths and the covered differential calls agree in success, return/revert bytes, log topics/data, balances and allowances. Requires the same logical state and sufficient gas. Gas costs, bytecode, raw storage and descriptive JSON ABI fields are not identical. Fuzzing is not universal proof.
Two-slot mapping storage layout	Holds	Compiler storage layout contains exactly slot 0 `balanceOf` and slot 1 `allowance`. Name, symbol, decimals and supply are constants occupying no storage slots.
Inability to receive any ETH	Does not hold	Ordinary ETH calls revert, but forced transfers or pre-funding can leave ETH in the contract. There is no withdrawal path; see I-01.

Supply and arithmetic argument

Let `S = 26_271_000_000 * 10**18`. Immediately after construction exactly one non-zero account has S, and every other mapping value is zero. For distinct sender and recipient, a successful transfer subtracts `v <= senderBalance` and adds the same v to the recipient. Non-negative balances and total sum S imply `recipientBalance + v <= S < 2**256`, so the addition cannot overflow. For a self-transfer, the addition reads the already-decremented mapping slot and restores the original balance. Reverted calls atomically restore earlier writes. `approve` writes no balances. Induction over successful calls therefore preserves the total, excluding arbitrary test storage manipulation and assuming correct compiler/EVM execution.

Independent execution evidence

Byte-identical ERC-20 behavior validated against OpenZeppelin v5.6.1 across all 480,000 tested operations. Return/revert bytes and event topics/data matched exactly, as did resulting balances and allowances. **20,000 sequences; 33 tests passed.**

The 33 tests comprise 31 independent boundary tests, one metadata comparison and one differential fuzz test running **20,000 sequences x 24 operations = 480,000 compared operations**; none failed or were skipped. The differential run completed in 24.98 seconds using Foundry v1.8.1 and the exact compiler/settings above. The audit harness was independently written.

Fuzz seed: `0xe91305ec970c14c38b4575a637f8192b5e380cf145a828aedd904f5934a08169`.

The reference uses OpenZeppelin Contracts v5.6.1, commit `5fd1781b1454fd1ef8e722282f86f9293cacf256`. Its constructor calls `ERC20("Harmony", "ONE")` and `_mint(recipient, 26_271_000_000 * 10**18)`. Tests use forge-std v1.15.0, commit `0844d7e1fc5e60d77b68e469bff60265f236c398`.

To reproduce the differential method, compile and deploy both implementations with the same recipient and settings, create six actors including zero, and execute 24 operations per generated sequence. Fund every non-zero actor with tokens and seed finite/unlimited grants through valid calls. Draw operations from transfer, approve and transferFrom, using eight amount modes: arbitrary uint256 values, zero, one, maximum uint256, balance, balance+1, bounded values and allowance. After every operation, compare success flags, complete return/revert bytes, event topics/data, all six actor balances and all 36 allowances, then check balance conservation. Validate each log emitter against its respective token address. Run 20,000 sequences with the seed above. These are the parameters used by the independent harness.

The 31 independent boundary tests cover constructor allocation/events, zero recipient, zero-value transfers, whole-supply self-transfer, insufficient balance, finite/unlimited approvals, overwrite, maximum-value rejection without panic, error precedence, rollback, self-transfer allowance consumption, unknown/admin selectors, nonpayable calls, a contract recipient and forced/pre-funded ETH. The contract-recipient test demonstrates token mechanics; it is not an audit of the eventual Safe.

The counts and results reported here come from the audit's own execution, rather than previously reported tests.

Compiler, EVM, ABI and verification

Compiler bugs and fork compatibility

The exact compiler binary was retrieved from the official Solidity distribution and SHA-256 verified as `d4abcf0b3e24b7948ddfd64c374d26c3214648717777790ecb936979054a129d`. The current retrieved Solidity bug registry was filtered for version 0.8.36 and checked against this source. No listed bug applies:

Version-range match	Why it does not apply
SOL-2026-5, MemoryByteArrayElementDeleteClearsWholeWord	Requires deleting an element of a memory bytes array under the legacy pipeline. The source has no `delete` or such array operation.
SOL-2026-4, SpillSlotCollisionAcrossMutualRecursion	Requires the IR pipeline and mutual recursion. `viaIR=false`; the call graph is non-recursive.

Other registry entries are fixed in 0.8.36 or earlier. The retrieved registry contains links dated September 10, later than this report's September 8 date. The analysis uses that retrieved dataset without treating the links as evidence of disclosure before the report date. Recheck the current registry before deployment. Official release, binary index, known-bugs registry.

Prague/Pectra activated on Ethereum mainnet on May 7, 2025; Osaka/Fusaka followed on December 3, 2025. Prague remains a valid target for this source. A later target is not required for its correctness; Amsterdam is experimental in the reviewed compiler documentation. Decoded runtime and creation instructions were inspected, excluding pushed data and the CBOR trailer. Runtime uses `PUSH0` and `MCOPY`, already supported on mainnet; no later-fork instruction is required. The constructor's embedded runtime is data copied into the deployed account. Ethereum fork history, Solidity EVM targets, MCOPY, PUSH0.

ABI and constructor compatibility

`string` constant getters return standard dynamic ABI strings, decimals returns standard ABI-encoded `uint8`, and `external` visibility does not change selectors or wire encoding relative to `public`. The compiled constant getters are `view`, as in the reference. The JSON ABI is not textually identical: anonymous mapping key/output names and descriptive fields such as internal types may differ. There is no identified encoding concern for ordinary wallets, explorers, DEX routers, bridges or custodians; individual integrations were not run. ABI specification.

The constructor's `Transfer(address(0), recipient, totalSupply)` is the standard mint signal for indexers. A contract recipient, including a Safe, receives tokens by a storage assignment; there is no receiver callback or reentrancy. Non-zero validation does not prove that the address is the intended treasury or can control the tokens. Verify the deployed Safe, chain and address before deployment; configuration is outside scope. ERC-20.

Metadata and size

Preserve the exact source bytes, source-unit name, compiler version, complete Standard JSON settings, metadata and constructor encoding for explorer verification. The reviewed source includes a supply rationale comment at line 8. Comments do not alter transfer logic but can change the metadata hash and therefore full bytecode. Solidity metadata specification.

Standalone build artifact	Independently reproduced value
Standard JSON source key	`HarmonyONEToken.sol`
Init code, excluding constructor argument	1,702 bytes
Init-code keccak256	`0xe6f90879ee5efa3150f196a750905a2128405393c35abb0f5ad3d16c49c4a047`
Runtime	1,465 bytes
Runtime keccak256	`0x277f937d07613df41830d132315454a36da1eb96d4b07e65998141ebc32613ba`
Runtime excluding metadata and length trailer	1,412 bytes
Runtime keccak256 excluding metadata	`0xaa1a0c7a8ef63f549bde669ce5f33b8e87abe82dff96bb8a0ed74b2f007e9374`

For comparison, the previously supplied build fingerprints were:

Artifact	Previously supplied keccak256
Init code	`0xe779cb41bedac8298ec9ced3fe6d45cb5e882be65de2e7352a40e00897f9917d`
Runtime	`0xac649c8153c7ac30d0d90949a8463dccede463d93eac9af9b68fef815460fc0d`
Runtime excluding metadata	`0xaa1a0c7a8ef63f549bde669ce5f33b8e87abe82dff96bb8a0ed74b2f007e9374`

The independently compiled metadata-free runtime hash **matches the supplied fingerprint exactly**, as do the 1,702-byte init-code and 1,465-byte runtime sizes. The supplied full init/runtime hashes were **not reproduced**, including after testing source-unit-name and remapping variants. Exact full-build identity remains unverified; it must not be inferred from the logic hash. Resolve the full-hash discrepancy and record the complete compiler input and exact deployment build before deployment. This is an artifact-reproduction limitation, not evidence of a transfer-logic defect.

The default metadata tail is a 51-byte CBOR payload **plus a two-byte length trailer**, 53 bytes in total. Strip the decoded trailer length plus two when comparing logic hashes. A hash over only the last 51 bytes removed is not the metadata-free runtime hash. Full hashes are sensitive to source paths and settings; inspect any difference before relying on an artifact identity.

The measured runtime and init code are comfortably below Ethereum's 24,576-byte runtime and 49,152-byte init-code limits. No size or independently observed execution concern changes the source recommendation. EIP-170, EIP-3860.

An isolated Prague Anvil deployment consumed **397,960 receipt gas**. The measurement used a non-zero constructor recipient, seeded an existing holder with 1,000 base units, and submitted each subsequent operation in a separate transaction:

Operation	Amount / initial condition	Receipt gas
Transfer to an existing holder	10 base units; recipient already holds 1,000	34,392
Transfer to a new holder	10 base units; recipient balance is zero	51,504
New approval	Set a previously zero allowance to 1,000	46,272
Finite-allowance transferFrom	Move 10 base units to the existing holder; allowance starts at 1,000	40,388
Unlimited-allowance transferFrom	Move 10 base units to the existing holder after setting allowance to maximum uint256	37,152

Receipts include transaction and calldata costs; deployment also includes creation costs and code deposit. These figures depend on addresses, calldata, state and compiler output, and are not execution-only estimates. A previously supplied deployment figure of 397,936 gas was not reproduced under this measurement. No comparison against the original gas workload or an identically measured OpenZeppelin deployment was performed.

Accepted design tradeoffs and limits

The usual approval replacement race, unlimited-allowance exposure, irrecoverable token transfers to inaccessible recipients, and absence of pause, rescue, permit and admin controls are intentional or standard behaviors shared with the selected reference. They are not reported as new vulnerabilities. Zero-caller simulations, finite gas differences and raw state introspection should not be conflated with ordinary ABI-level ERC-20 equivalence.

The conclusions apply to the identified source, compiler settings and reviewed behavior. They do not establish formal verification, integration compatibility in every system, or the identity and configuration of a future deployment.